GAMES HUB

MINOR PROJECT REPORT

MASTER OF COMPUTER APPLICATIONS

3RD SEMESTER

Submitted by

Kshitij Gupta
Enrollment No.: U2449028
Batch Year: 2024-2026

Under the supervision of

Dr. Farha

Centre of Computer Education & Training
Institute of Professional Studies
University of Allahabad, Prayagraj
Uttar Pradesh - 211002

ACKNOWLEDGEMENT

I would like to take this opportunity to express my gratitude to everyone who has supported us in completing this project.

Firstly, I would like to express my sincere gratitude to **Dr. Nilesh Anand Srivastava,** the coordinator of the **Centre of Computer Education** at the **University of Allahabad**, for his invaluable guidance and support throughout my project work.

Foremost, I would like to thank our project incharges, **Dr. Sarika Yadav and Ms. Sunita Tripathi**, for their invaluable guidance, feedback, and encouragement throughout the project. I would also like to thank all entire CCET staff who have constantly resolved our domain-specific queries throughout the project. Their expertise and experience have been instrumental in shaping our understanding of the subject and developing our research skills.

I extend my sincere thanks to my respected project guide **Dr. Farha** for her invaluable guidance, constant support, and continuous motivation throughout the development of this project. Her encouragement, insightful suggestions, and expertise have been instrumental in the successful completion of this work.

Last but not the least I am also grateful to my family and my friends for their constant support and motivation, which has kept me going throughout the project. Their belief in me and my abilities has been a source of inspiration and encouragement.

SELF-DECLARATION

I hereby declare that the project entitled "**Games Hub"** is my original work and has not been submitted, in part or in full, for any other degree. All the information and data presented in the report are authentic and have been surely acknowledged.

I confirm that all the sources consulted during my work have been duly acknowledged in the reference section, and the report does not contain any plagiarized content. I understand that any act of plagiarism, including copying or paraphrasing the work of others without proper citation, is considered academic misconduct and can lead to severe penalties. I also understand that any misrepresentation or falsification of information can lead to the rejection of my project report and can affect my academic standing.

DATE: 03/12/2025 **Kshitij Gupta**

Place: CCET, IPS, UoA, Prayagraj (Enrollment No.- U2449028)

CERTIFICATE

This is to certify that **Kshitij Gupta**, student of the **Master of Computer Applications (Semester –III)** of the Centre of Computer Education, IPS, UoA, has completed his work in the project entitled **"Games Hub**" under my guidance in the academic session (2025–26).

He has taken proper care and shown utmost sincerity in the completion of this project. I certify that his work on this project is up to my expectations and as per the guidelines. This project is original and is never submitted somewhere else for the same purpose.

DATE: 03/12/2025 **Dr. Fraha**

Place: CCET, IPS, UoA, Prayagraj (Project Supervisor)

TABLE OF CONTENTS

S.No.	Contents	Page No.
1.	Acknowledgment……..	I
2.	Self-Declaration……	II
3.	Certificate……..	II
4.	Synopsis……	1-15
5.	Project Report……..……	18-44
6.	Introduction……..	16-18
	6.1 Problem Definition……	17
	6.2 Motivation…….	18
7.	Objectives……	19
8.	Requirement Analysis and Specification……	20-21
	8.1 Functional and Non-Functional requiremts….	20
	8.2 Hardware and Software Requirements…….	21
9.	System Design……	22-23
	9.1 Data Flow Diagram……...	22
	9.2 Use Case Diagram…….	23
10.	Module Specification……	24
11.	Application Interface……..	25-32
12.	Code……....……	33-39
13.	Conclusion……...	40
14.	Future Scope……	41
15.	Milestones……	42
16.	Meeting with Supervisor……..	43
17.	References……	44

SYNOPSIS

PROJECT REPORT

1. Introduction

Project Name: *Games Hub: A Freemium Mobile Mini-Games Application*
Category: Mobile Application

Mobile gaming has emerged as one of the most dominant forms of digital entertainment, driven by the increasing demand for quick, engaging, and easily accessible experiences on smartphones. Despite this growth, casual gamers often encounter challenges such as the need to install multiple applications to access different mini-games, resulting in storage overhead, fragmented progress tracking, and a disjointed user experience.

To address these limitations, the proposed project, Games Hub, introduces a freemium mobile application developed using Flutter (Dart). The application consolidates multiple lightweight offline mini-games into a single platform, thereby enhancing convenience and accessibility. Under the freemium model, free users are provided with several offline games, while Pro (premium) users gain access to exclusive titles and advanced features through an integrated in-app upgrade system.

The application incorporates secure user authentication, real-time cloud database management, and a simulated payment gateway, making it not only a source of entertainment but also a practical demonstration of contemporary mobile app monetisation strategies. Furthermore, Games Hub exemplifies modern mobile development practices, including cross-platform UI design, offline-first architecture, and seamless data synchronisation.

In essence, Games Hub seeks to deliver an all-in-one mini-games environment that eliminates the inefficiencies of fragmented gaming applications, while simultaneously serving as a case study in the implementation of scalable and monetizable mobile solutions.

2. Problem Definition

With the rise of mobile gaming, users frequently download and switch between separate applications to access different mini-games. This results in a fragmented gaming experience, increased storage usage and lack of continuity in gameplay. Many existing mini-game applications do not offer secure user authentication, personalised statistics or progress tracking, which makes it difficult for users to maintain their achievements across multiple games. Additionally, there is no centralised platform where both free and premium games are available under a single interface, reducing user convenience and engagement.

Most gaming platforms today rely heavily on continuous internet connectivity, which restricts accessibility for users who wish to play offline or have limited network availability. At the same time, many apps do not support scalable database management, preventing the storage of user profiles, game history and leaderboard rankings in a unified manner. Current applications also lack a well-defined freemium model that allows users to start with free games and later upgrade seamlessly to access premium content through an integrated payment system.

There is therefore a clear need for a single, unified gaming platform that houses multiple offline mini-games, ensures secure login, tracks player progress and supports premium upgrades through a simple payment workflow. The solution must provide smooth navigation, reduced dependency on multiple apps and a balanced experience for both free and pro users. Addressing these challenges will result in an efficient and engaging mobile gaming system that enhances usability, accessibility and monetisation through a compact all-in-one application.

3. Motivation

Casual mobile games have become an essential source of quick entertainment, mental refreshment and short interactive engagement for users during their daily routines. However, most users who enjoy mini-games often face difficulty in managing multiple separate applications, which not only consumes device storage but also disrupts the continuity of the gaming experience. Despite the popularity of casual gaming, there is still a noticeable gap in the market where a unified platform offering multiple offline mini-games, secure login, progress tracking and consistent user interface is rarely available.

The proposed application, **Games Hub**, aims to address this challenge by providing an all-in-one gaming platform that bundles numerous mini-games under a single application. It ensures secure user authentication, maintains gameplay statistics and offers a structured tracking mechanism for user performance. One of the major motivations for this project is its freemium model, which allows users to begin with free access and later upgrade to Pro to unlock exclusive premium content through integrated payment processing.

This project not only enhances user convenience by preventing the need to install multiple apps, but also serves as a valuable learning opportunity for implementing real-world concepts such as game logic, database integration, authentication, offline functionality and monetisation models. Thus, **Games Hub** stands as a practical and engaging solution that improves the overall gaming experience while strengthening development knowledge and technical skills.

4. Objective

To develop a unified cross-platform mobile gaming application offering multiple offline mini-games with secure authentication, scalable user data handling, and a freemium upgrade model through a smooth and intuitive interface.

5. Requirement Analysis and Specification

Functional Requirements:

The Games Hub application is designed to provide multiple offline mini games through a unified mobile platform and therefore requires well-structured functional components. The system must allow users to register and authenticate securely through email and password, enabling protected access to their profiles. Each user should have a personalized profile where gameplay statistics such as total matches played, wins, losses and game progress are stored and retrieved dynamically. The application must provide access to several free mini games while restricting premium ones until the user upgrades to a Pro subscription. A smooth dashboard-based navigation system should allow users to switch easily between games, view their achievements and update their profile details. Offline gameplay is a core requirement, ensuring that games function without internet and sync data to the database once the connection is restored. For monetization, the platform must integrate a payment gateway in test mode, enabling users to purchase Pro access, after which exclusive games should unlock instantly. Additionally, game logic, leaderboard tracking and score management must function accurately to maintain fair and engaging gameplay.

Non-Functional Requirements:

Beyond core functionality, the system must meet several non-functional requirements to ensure reliability and performance. The application should load quickly, respond smoothly to user input and operate efficiently on both Android and iOS devices using a single Flutter codebase. It must be scalable enough to handle a growing number of users, database requests and gameplay records without performance degradation. Security is a priority, requiring encrypted storage for sensitive data and authentication measures that prevent unauthorized access. Payment transactions must be processed securely through a protected and verified gateway. The system must also be portable, allowing deployment across multiple devices with varying hardware capacity. Usability plays a key role, ensuring an intuitive interface with easy navigation and visually appealing design for seamless user interaction. The application should also maintain high reliability, ensuring crash-free operations and consistent data handling even during offline-to-online sync. In addition, the project should utilize Firebase services for backend operations, cloud storage, and analytics to track user behavior and improve the application's overall effectiveness.

6. Hardware and Software Requirements

1. Hardware Requirements:

Category	Minimum Requirement	Recommended Requirement
Processor	Intel i3 (10th Gen)	Intel i5/i7 (11th Gen) +
RAM	4 GB	8 GB or higher
Storage	20 GB free space	50 GB SSD or higher
Graphics	Integrated GPU	Dedicated GPU (for smooth emulation)
Mobile Device (Testing)	Android phone (8.0+)	Latest Android/iOS device
Internet Connection	5 Mbps	20 Mbps or higher

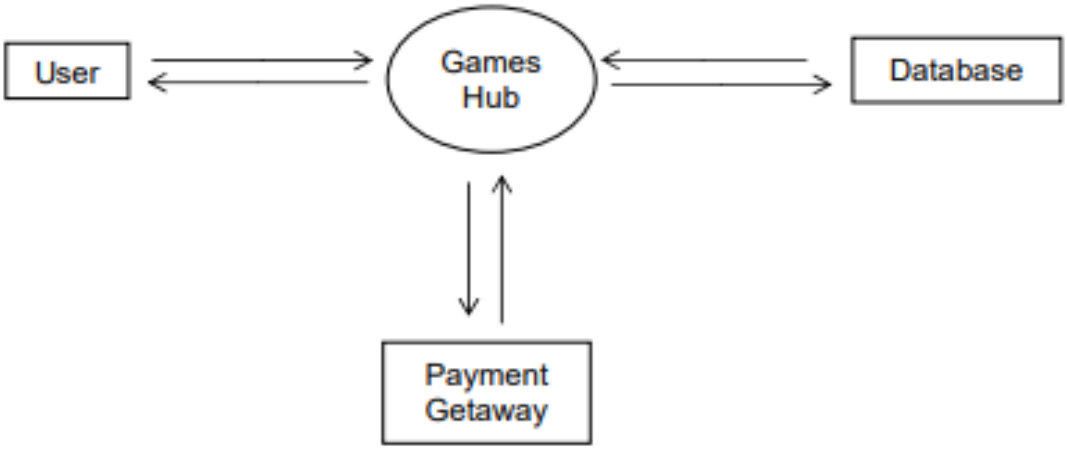
2. Software Requirements:

Category	Specification
Operating System	Windows 10 / 11, macOS, or Linux (for development)
Mobile Platforms	Android 8.0+ / iOS 13+
Programming Language	Dart (Flutter SDK)
Frameworks	Flutter
Database	Firebase Firestore
Authentication	Firebase Authentication
Payment Gateway	Razor-pay (Sandbox & Live mode)
Tools/IDE	VS Code and Android Studio
Version Control	Git + GitHub

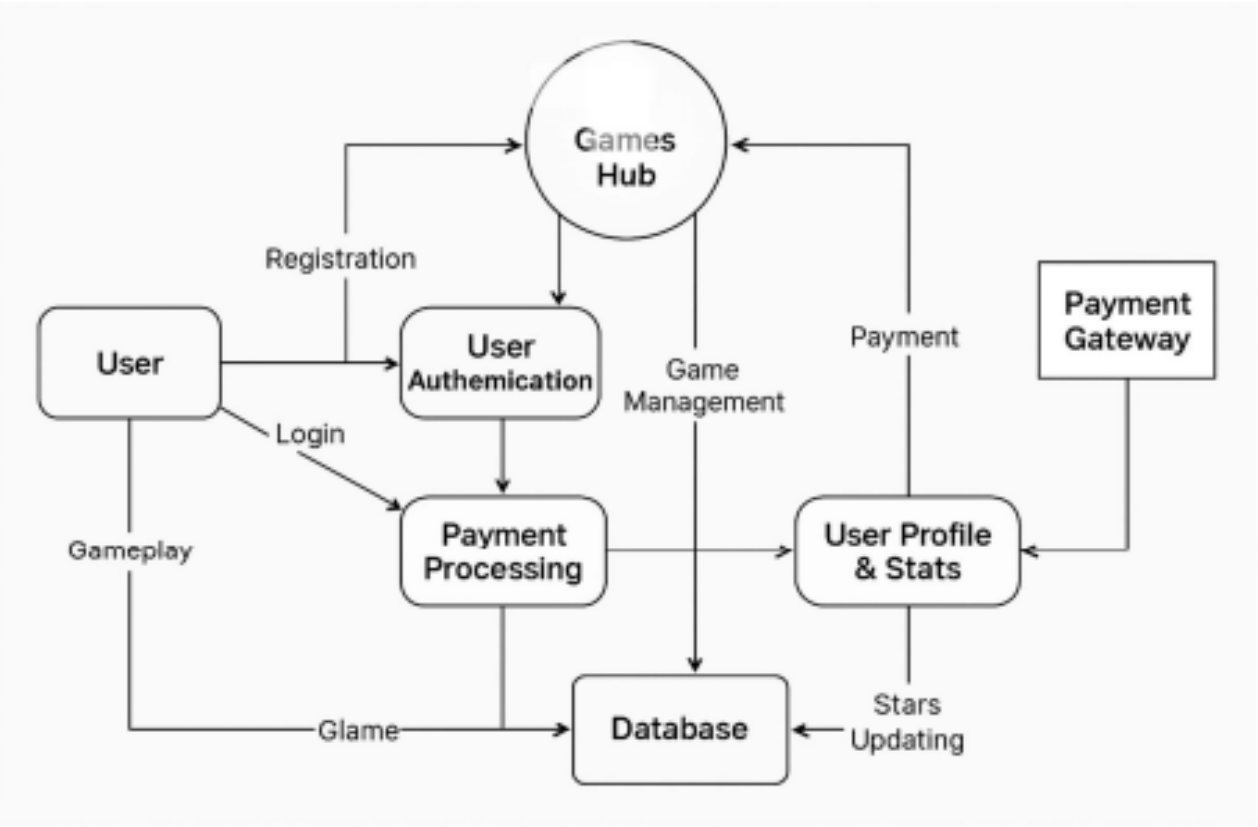
7. System Design

Data Flow Diagram:

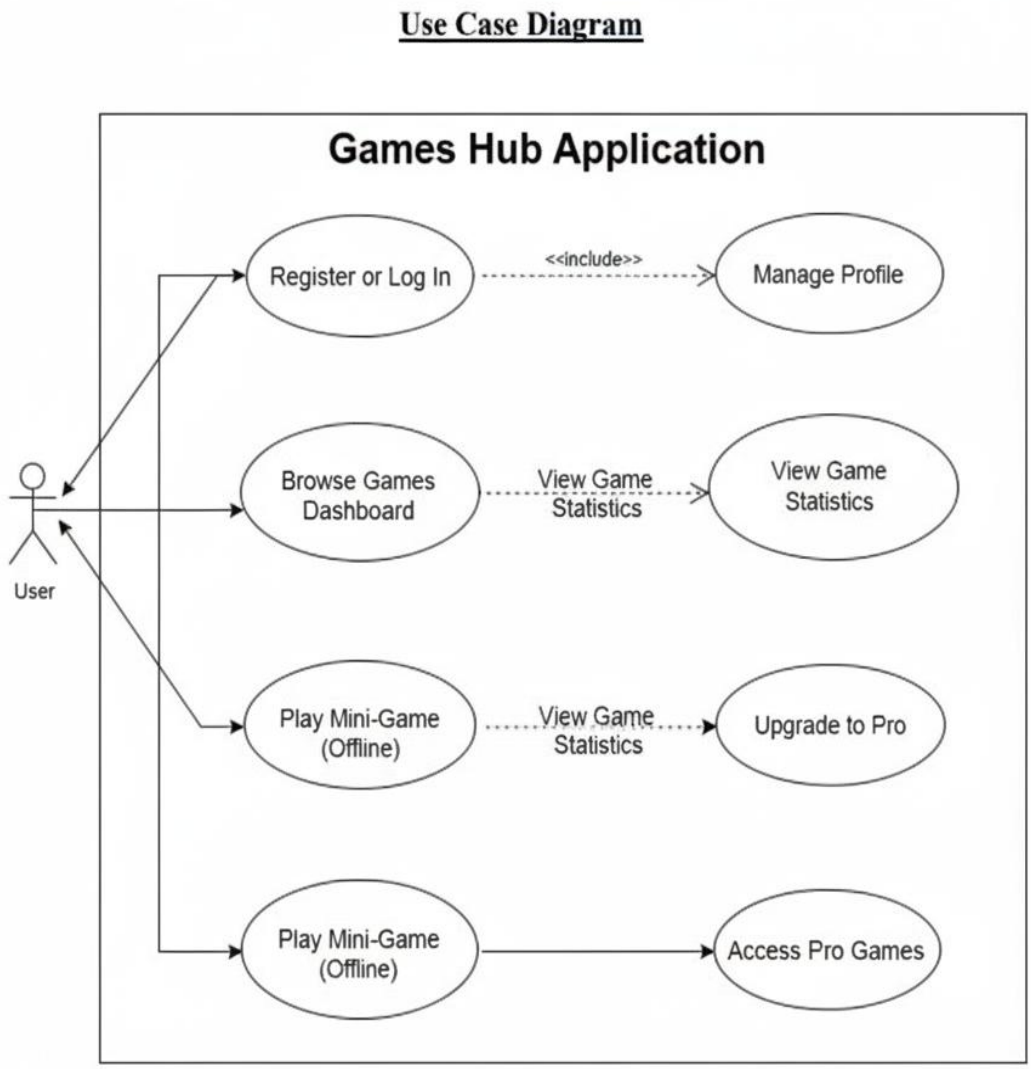
Level 0:



Level 1:



8. Use Case Diagram



9. MODULE SPECIFICATION

1. Authentication Module:
This module manages secure user login, registration, and password recovery using Firebase Authentication. It validates credentials, generates session tokens, and ensures safe access to app features.

2. User Profile Module:
The user profile module stores and updates personal details such as username, avatar, and account type (Free/Pro). It synchronizes profile data with the database and allows users to modify personal information anytime.

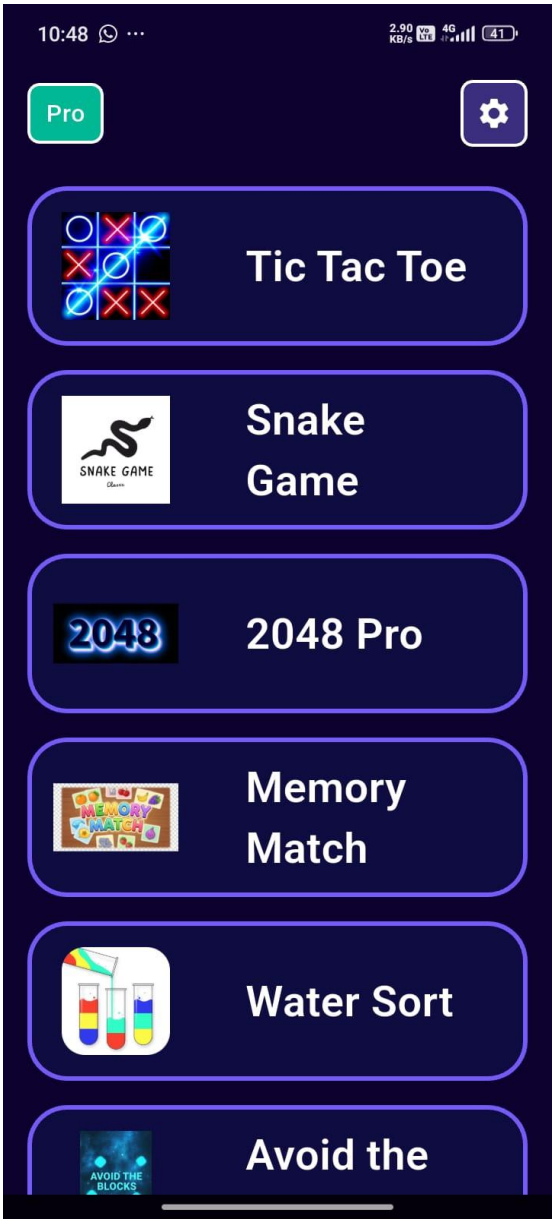
3. Leaderboard & Statistics Module:
This module records gameplay data including score, wins, and achievements, and displays global or personal ranking. It continuously updates progress and generates performance insights for users.

4. Individual Game Modules (Game Engine Wrappers):
Each game module contains its own logic, rules, UI layout, and scoring system. The engine handles input, gameplay flow, progress tracking, and final result output for every mini-game.

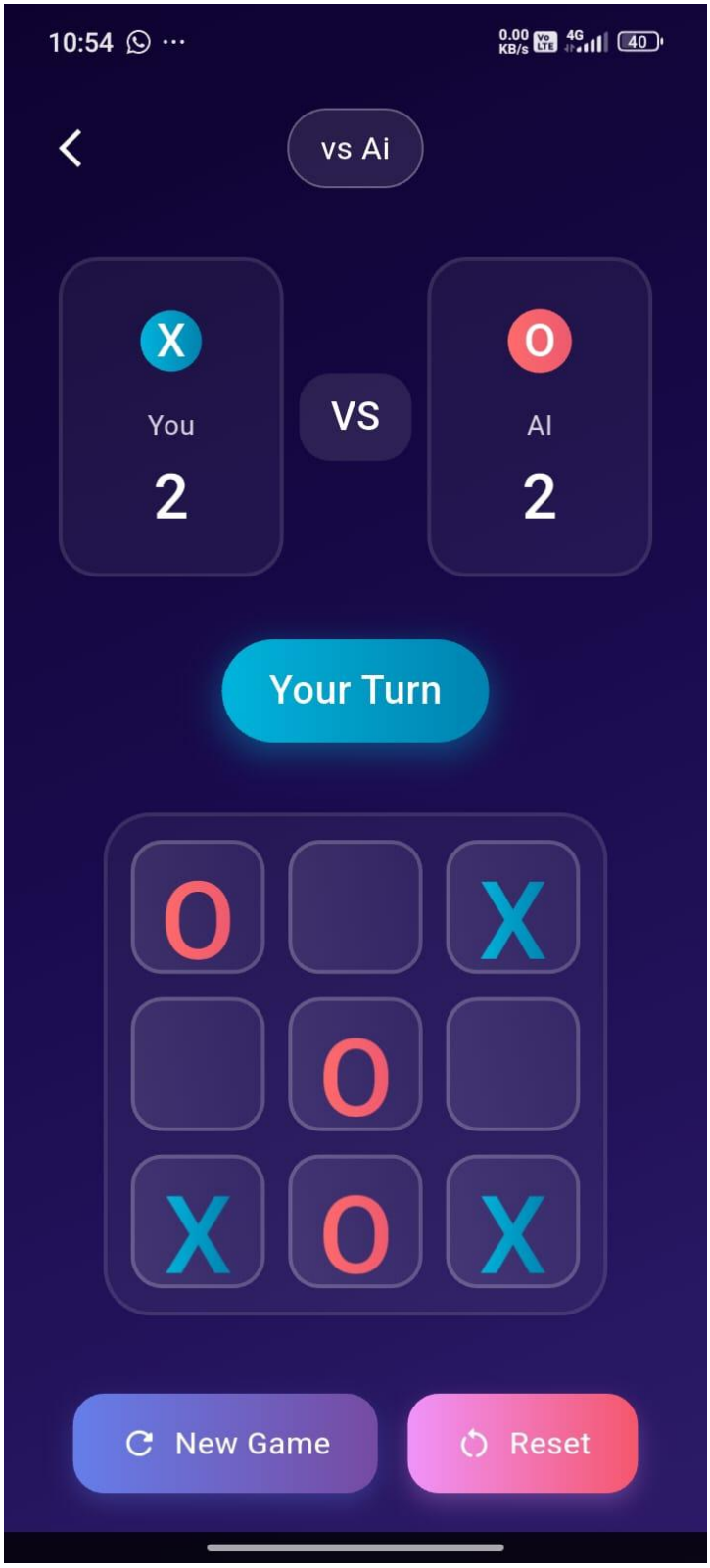
5. Payment & Upgrade Module:
The payment module manages Pro upgrade through Razor pay sandbox integration and unlocks premium games upon successful payment. It verifies transactions, updates account status, and stores purchase history securely.

10. Application Interface

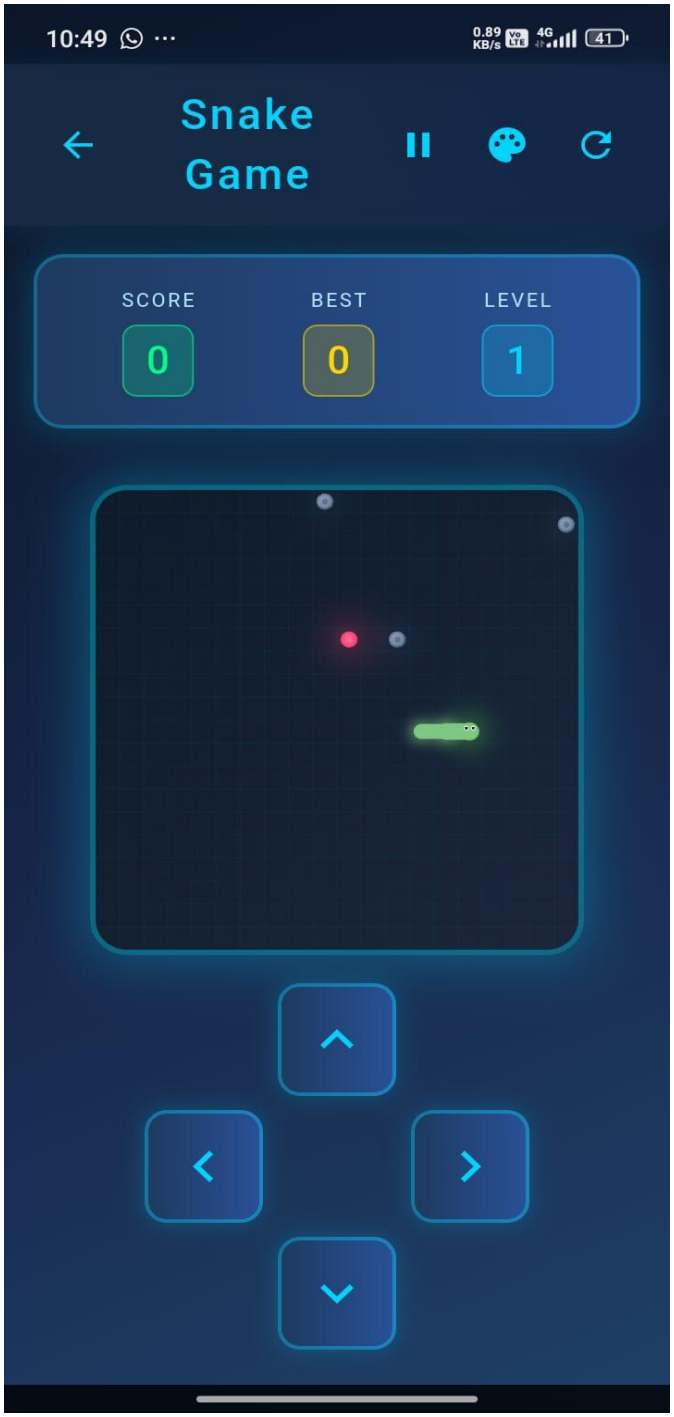
Home page:



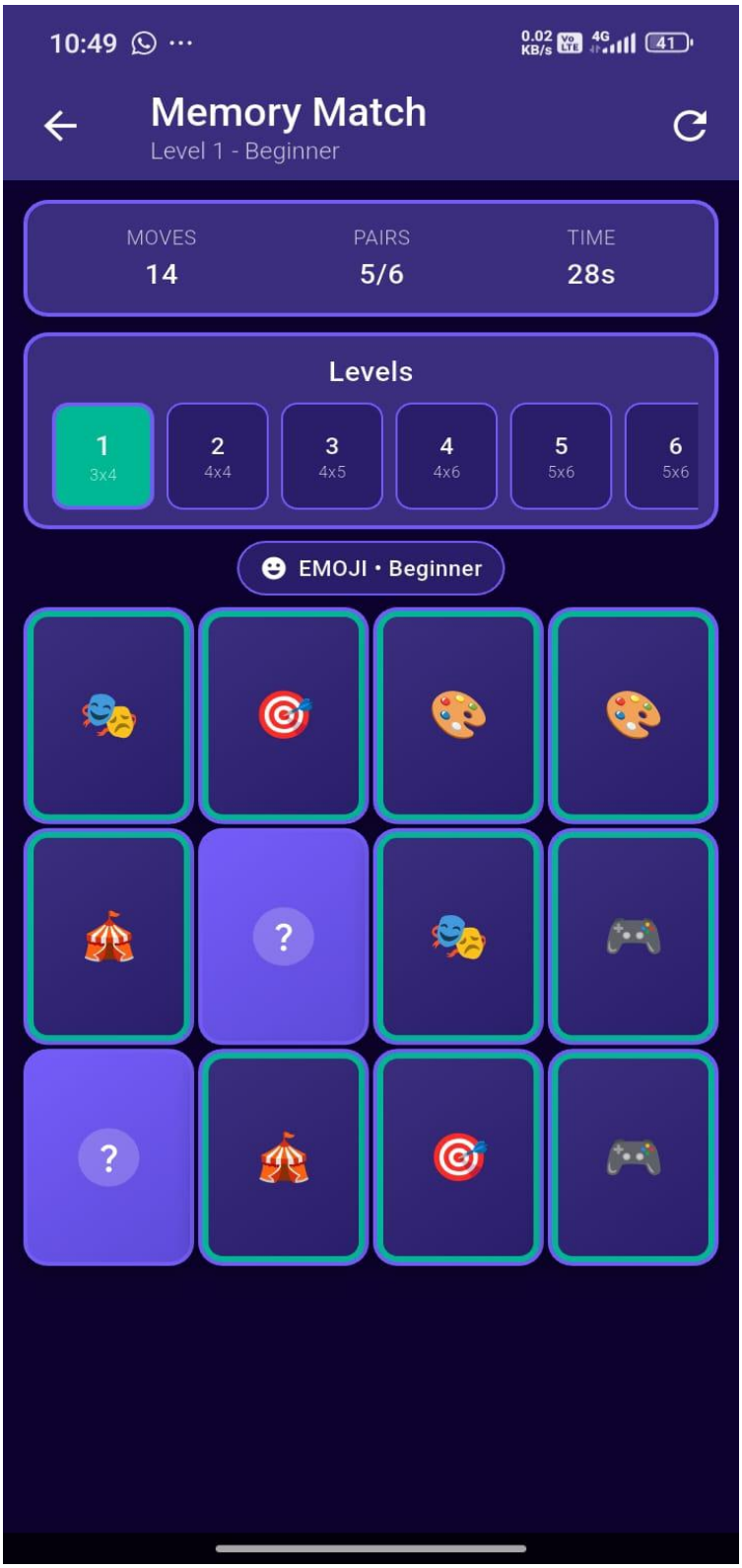
Tic Tak Toe Game:



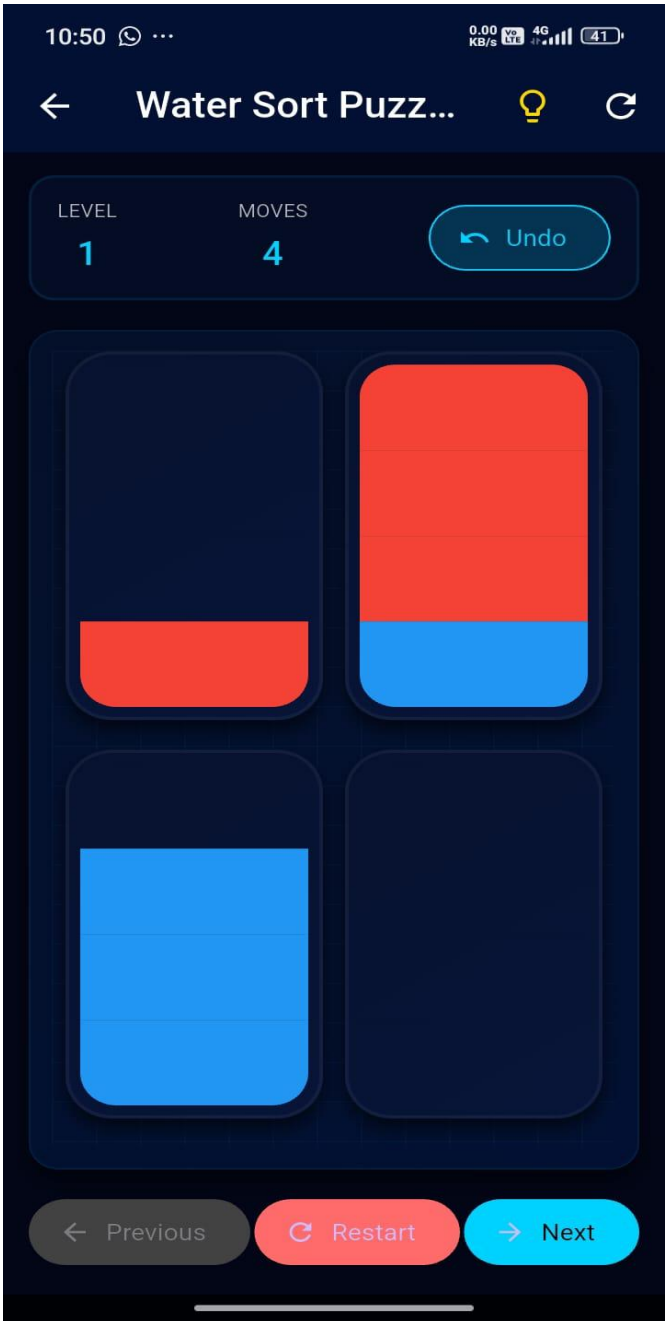
Snake Game:



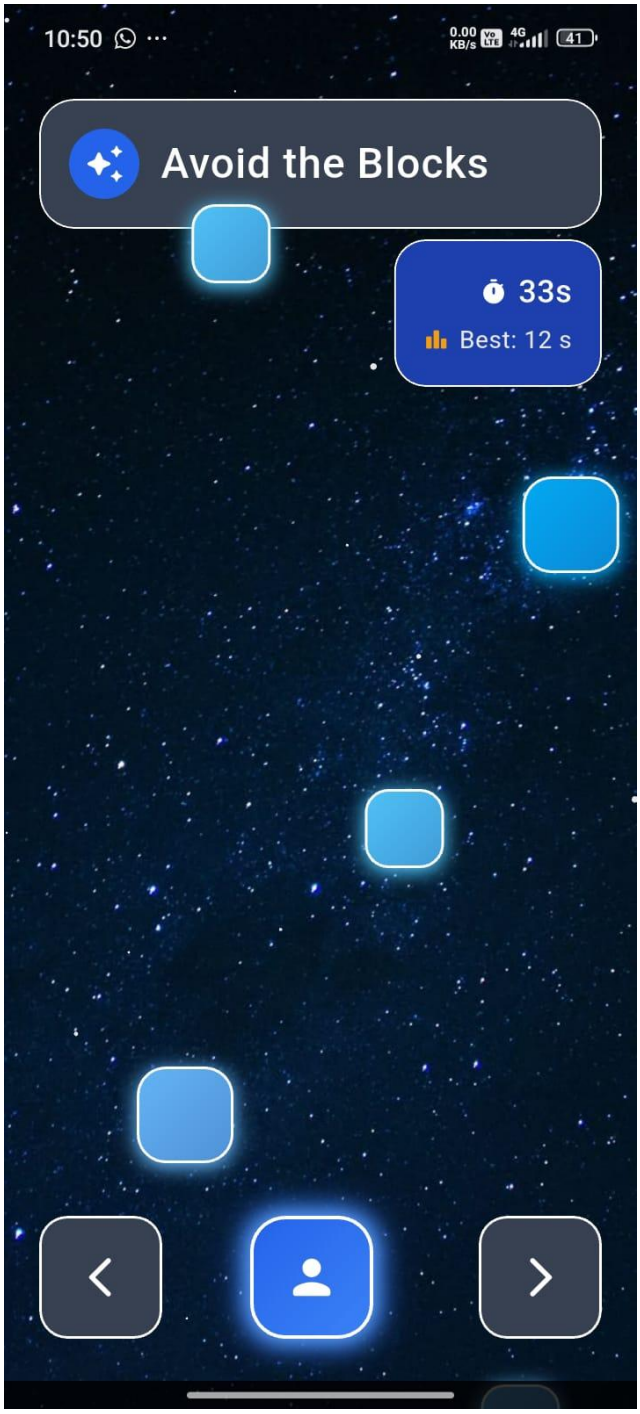
Memory Match Game:



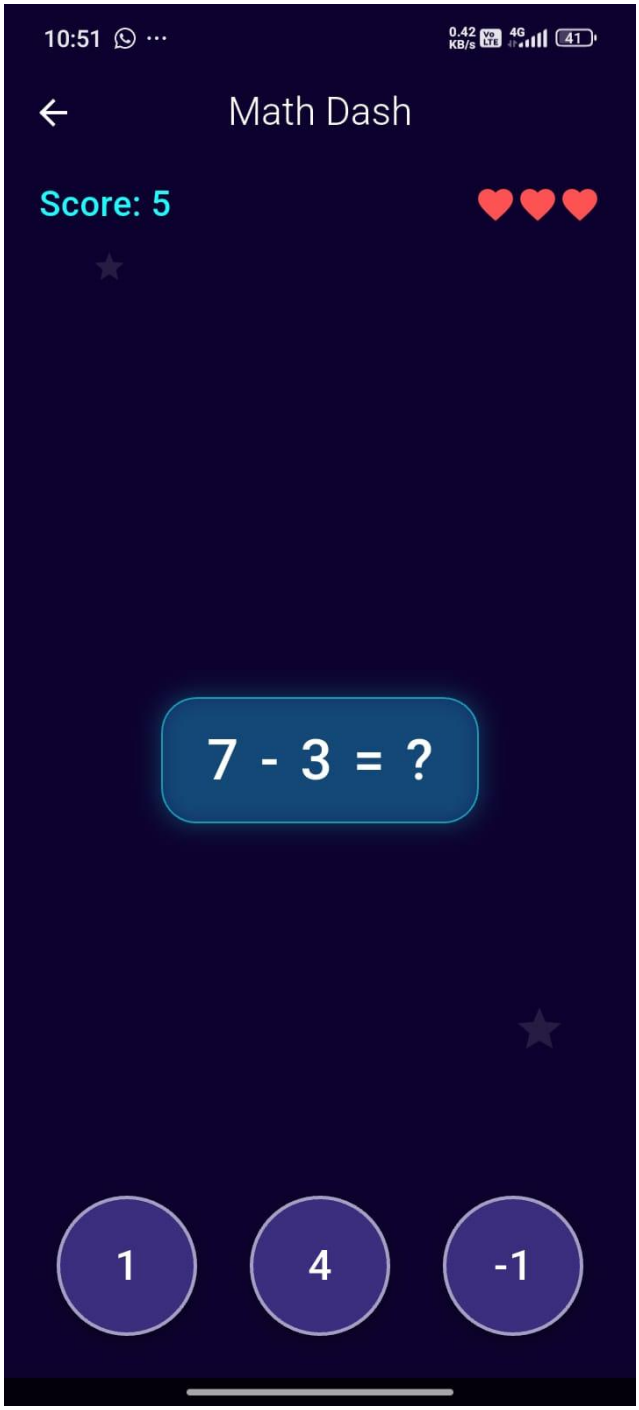
Water Sort Puzzle:



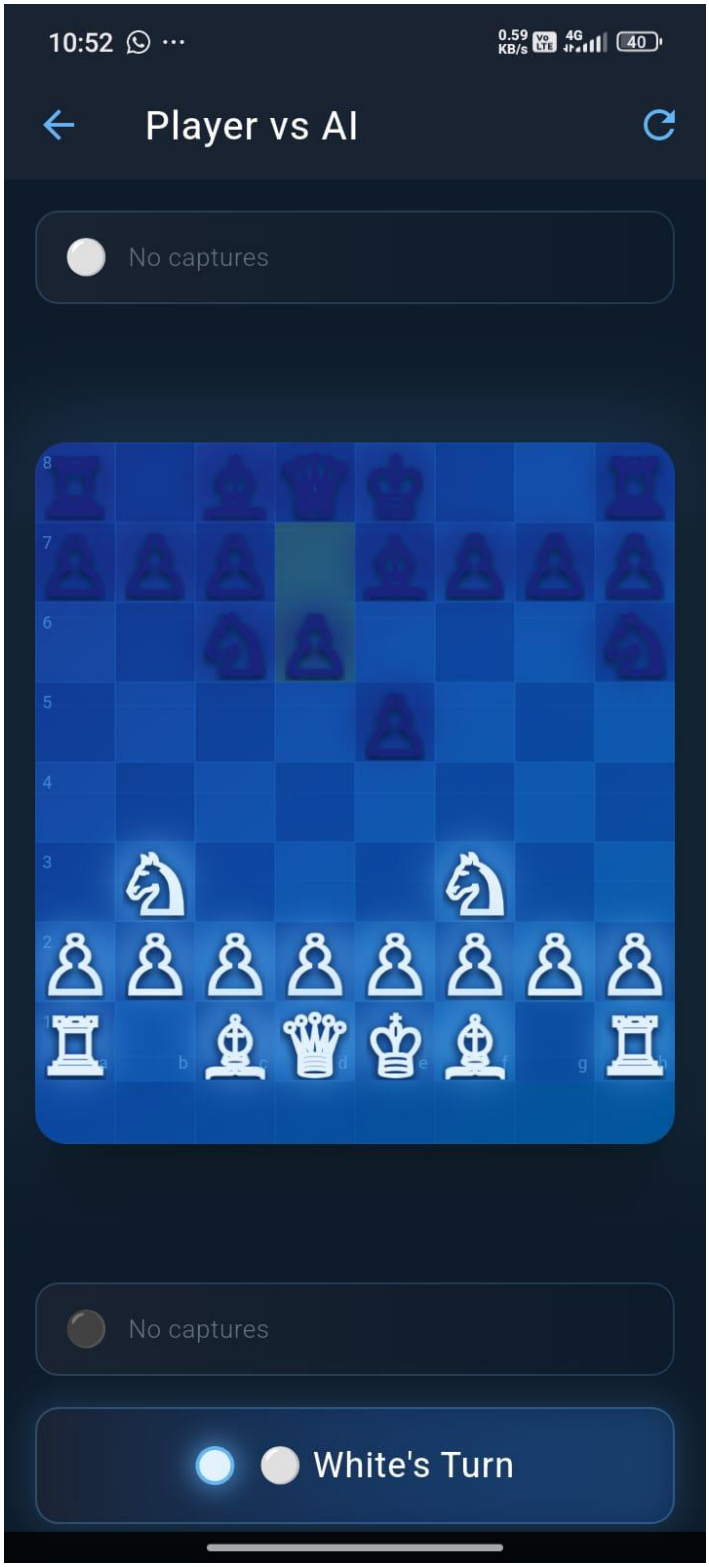
Avoid The Blocks:



Math Dash:



Chess Game:



11.Code

Main.dart:

```
import 'package:flutter/material.dart';
import 'screens/homes.dart';

void main() {
  runApp(const GamesHubApp());
}
```

Homes.dart:

```
import "package:flutter/material.dart";
// Importing individual game screens and common widgets
import "package:flutter_application_3/Games/TicTakToe/tictaktoe.dart";
import "package:flutter_application_3/Games/chess/chess_mode_selection.dart";
import 'package:flutter_application_3/Games/color_switch/color_switch_screen.dart';
import "package:flutter_application_3/Games/game_2048/game_2048_screen.dart";
import "package:flutter_application_3/Games/memory_match/memory_match.dart";
import "package:flutter_application_3/Games/snake_game/snake_game_screen.dart";
import "package:flutter_application_3/common/widgets/game_card.dart";
import 'package:flutter_application_3/Games/watersort/watersort.dart';
import 'package:flutter_application_3/Games/avoid_blocks/avoid_blocks_screen.dart';
import 'package:flutter_application_3/Games/math_dash/math_dash_screen.dart';

class GamesHubApp extends StatelessWidget {
  // Root widget of the app
  const GamesHubApp({super.key});

  @override
  Widget build(BuildContext context) {
    // MaterialApp provides material design visual structure
    return MaterialApp(
      title: 'Games Hub',
      theme: ThemeData(
        // Dark theme with primary purple color
        brightness: Brightness.dark,
        primarySwatch: Colors.deepPurple,
        scaffoldBackgroundColor: const Color(0xFF0D022F),
      ),
      home: const HomeScreen(), // Home screen of the app
      debugShowCheckedModeBanner: false, // Disable debug banner
    );
  }
}

class HomeScreen extends StatefulWidget {
```

```
  // Stateful widget for the main screen with dynamic state
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}
class _HomeScreenState extends State<HomeScreen> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        // Column to arrange top bar and game list vertically
        child: Column(
          children: [
            // Top bar with "Pro" label and Settings icon on opposite ends
            Padding(
              padding: const EdgeInsets.all(16.0),
              child: Row(
                mainAxisAlignment: MainAxisAlignment.spaceBetween,
                children: [
                  Container(
                    // Pro badge container with green background
                    width: 50,
                    height: 40,
                    decoration: BoxDecoration(
                      color: const Color(0xFF00B894),
                      borderRadius: BorderRadius.circular(8),
                      border: Border.all(color: Colors.white, width: 2),
                    ),
                    child: const Center(
                      child: Text(
                        'Pro',
                        style: TextStyle(
                          color: Colors.white,
                          fontWeight: FontWeight.bold,
                          fontSize: 16,
                        ),
                      ),
                    ),
                  ),
                  Container(
                    // Settings icon container with purple background
                    padding: const EdgeInsets.all(8),
                    decoration: BoxDecoration(
                      color: const Color(0xFF3B2E7E),
                      borderRadius: BorderRadius.circular(8),
                      border: Border.all(color: Colors.white, width: 2),
                    ),
                    child: const Icon(
                      Icons.settings,
```

```
          color: Colors.white,
          size: 24,
        )    ),],),),),
// Expanded ListView to show game cards scrollably
Expanded(
  child: Padding(
    padding: const EdgeInsets.symmetric(horizontal: 16.0),
    child: ListView(
      children: [
        const SizedBox(height: 10),

        // Game cards for each game with image, title and onTap handler

        GameCard(
          title: 'Tic Tac Toe',
          assetPath: 'assets/tictactoe.png',
          onTap: () => _navigateToTicTacToe(context),
        ),

        const SizedBox(height: 16),

        GameCard(
          title: 'Snake Game',
          assetPath: 'assets/snakegame.png',
          hasNewTag: true,
          onTap: () => _navigateToSnakeGame(context),
        ),

        const SizedBox(height: 16),

        GameCard(
          title: '2048 Pro',
          assetPath: 'assets/2048_logo.png',
          hasNewTag: false,
          onTap: () => _navigateTo2048(context),
        ),

        const SizedBox(height: 16),

        GameCard(
          title: 'Memory Match',
          assetPath: 'assets/memory_match.png',
          hasNewTag: false,
          onTap: () => _navigateToMemoryMatch(context),
        ),

        const SizedBox(height: 16),

        GameCard(
          title: 'Water Sort',
```

```
            assetPath: 'assets/watersort.png',
            hasNewTag: true,
            onTap: () => _navigateTowatersort(context),
          ),

          const SizedBox(height: 16),

          GameCard(
            title: 'Avoid the Blocks',
            assetPath: 'assets/avoid_blocks.jpeg',
            hasNewTag: true,
            onTap: () => _navigateToAvoidBlocks(context),
          ),

          const SizedBox(height: 16),

          GameCard(
            title: 'Math Dash',
            assetPath: 'assets/math_dash.png',
            hasNewTag: true,
            onTap: () => _navigateToMathDash(context),
          ),

          const SizedBox(height: 16),

          GameCard(
            title: 'Chess',
            assetPath: 'assets/chess.jpg',
            hasNewTag: true,
            onTap: () => _navigateToChessGame(context),
          ),

          const SizedBox(height: 16),

          GameCard(
            title: 'Color Switch',
            assetPath: 'assets/color_switch/ball.png',
            hasNewTag: false,
            onTap: () => _navigateToColorSwitch(context),
          ),

          const SizedBox(height: 30),
        ],
      ),
    ),
  ),
 ],
),
),
);
```

```
}

// Navigation methods to push respective game screens on tap events

void _navigateToTicTacToe(BuildContext context) {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => const TicTacToeScreen()),
  );
}

void _navigateToSnakeGame(BuildContext context) {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => const SnakeGameScreen()),
  );
}

void _navigateTo2048(BuildContext context) {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => const Game2048Screen()),
  );
}

void _navigateToMemoryMatch(BuildContext context) {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => const MemoryMatchScreen()),
  );
}

void _navigateTowatersort(BuildContext context) {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => const WaterSortScreen()),
  );
}

void _navigateToAvoidBlocks(BuildContext context) {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => const AvoidBlocksScreen()),
  );
}

void _navigateToMathDash(BuildContext context) {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => const MathDashScreen()),
```

```
    );
  }

  void _navigateToChessGame(BuildContext context) {
    Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => const ChessModeSelection()),
    );
  }

  void _navigateToColorSwitch(BuildContext context) {
    Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => const ColorSwitchScreen()),
    );
  }
}
```

Game_card.dart:

```
import 'package:flutter/material.dart';

class GameCard extends StatefulWidget {
  // Title of the game displayed on the card
  final String title;

  // Path to the image asset for the game icon
  final String assetPath;

  // Whether the "NEW" tag should be shown (removed here)
  final bool hasNewTag;

  // Callback when the card is tapped
  final VoidCallback onTap;

  const GameCard({
    super.key,
    required this.title,
    required this.assetPath,
    this.hasNewTag = false,
    required this.onTap,
  });

  @override
  State<GameCard> createState() => _GameCardState();
}

class _GameCardState extends State<GameCard> {
```

```
@override
Widget build(BuildContext context) {
  return GestureDetector(
    // Trigger the onTap callback when the card is tapped
    onTap: widget.onTap,
    child: Stack(
      clipBehavior: Clip.none,
      children: [
        Container(
          // Main container for the card styling
          height: 105,
          decoration: BoxDecoration(
            color: const Color.fromARGB(190, 15, 15, 70), // Dark blue background
            borderRadius: BorderRadius.circular(26), // Rounded corners
            border: Border.all(
              color: const Color(0xFF755CF9), // Purple border color
              width: 3,
            ),
          ),
          child: Row(
            children: [
              // Container for the game image icon
              Container(
                width: 110,
                height: 110,
                padding: const EdgeInsets.all(14.0),
                child: Image.asset(
                  widget.assetPath,
                  fit: BoxFit.contain, // Contain fit to keep image aspect ratio
                ),
              ),

              // Expanded widget for the game title text
              Expanded(
                child: Padding(
                  padding: const EdgeInsets.symmetric(horizontal: 30),
                  child: Text(
                    widget.title,
                    style: const TextStyle(
                      color: Colors.white,
                      fontSize: 28,
                      fontWeight: FontWeight.w800,
                    ),
                  ),
                ),
              ),
            ],
          ),
        ),
      ],}
```

12. Conclusion

The Games Hub application successfully demonstrates a unified mini-gaming platform that eliminates the need for multiple separate game installations and provides users with a smooth, engaging, and feature-rich experience. By integrating secure user authentication, a scalable database, offline gameplay, leaderboard tracking, and a freemium upgrade model, the system delivers both convenience and enhanced accessibility to casual gamers. The combination of multiple mini games in a single app, along with Pro-level unlockable content, not only improves user experience but also reflects a practical implementation of real-world monetization and modern mobile development practices.

Overall, this project represents a strong step toward creating an efficient, enjoyable, and scalable gaming environment, while also contributing technical learning in areas such as Firebase integration, payment processing, and Flutter-based multi-module design.

13. Future Scope

Addition of More Mini-Games: More casual and strategy-based games can be integrated to make the platform richer and more diverse for different age groups and interests.

Online & Multiplayer Mode: A future upgrade can introduce online multiplayer gaming, allowing users to compete with friends or random global players in real-time.

Social Features & Community Interaction: Friend lists, chat system, game invites, clans/groups, and user profiles can be expanded to build a gaming community inside the application.

AI-Enhanced Gameplay: Artificial Intelligence can be used to provide difficulty levels, smart opponents, and personalized gameplay recommendations.

Improved UI/UX & Accessibility: More visually appealing themes, sound enhancements, animations, and accessibility options (voice assist, colour-blind mode) can improve usability.

Global Tournaments & Leaderboard Expansion: Weekly competitions, ranked seasons, and achievement badges can increase user engagement and long-term retention.

App Store Deployment & Scaling: Launching officially on Google Play Store and Apple App Store can scale the project to real users, followed by continuous optimization and updates.

14. Milestones

Sr. No.	Project Activity	Estimated Start Date	Estimated End Date
1.	Project Allotment	10/08/2025	20/08/2025
2.	Synopsis Creation	21/08/2025	30/08/2025
3.	Project Synopsis Presentation	16/10/2025	17/10/2025
4.	Frontend Development	10/09/2025	15/10/2025
5.	Back-end Development	15/10/2025	30/10/2025
6.	Integration and Testing	01/11/2025	05/11/2025
7.	Documentation	20/11/2025	28/11/2025
8.	Final Presentation	4/11/2025	4/11/2025

15. Metting with Supervisor

Date of the Meeting	Mode	Comments by the Supervisor	Signature of the Supervisor
10/08/2025	Offline		
31/08/2025	Offline		
16/10/2025	Offline		
27/11/2025	Offline		
28/11/2025	Offline		

16. References

1. Google Firebase. (2024). *Firebase Authentication Documentation.* Retrieved from https://firebase.google.com/docs/auth

2. Google Firebase. (2024). *Cloud Firestore Documentation.* Retrieved from https://firebase.google.com/docs/firestore

3. Razor pay. (2024). *Razorpay Payment Gateway Integration for Flutter.* Retrieved from https://razorpay.com/docs/payments/payment-gateway/flutter-integration/

4. Flutter.dev. (2024). *Build Apps for Any Screen with Flutter.* Retrieved from https://flutter.dev

5. Statista. (2024). *Mobile Gaming Worldwide - Statistics & Facts.* Retrieved from https://www.statista.com/topics/1906/mobile-gaming/

6. Google Developers. (2024). *Implementing Offline Functionality in Flutter Apps.* Retrieved from https://developer.android.com/topic/performance/offline